

Embedded Systems Homework #7

Nidhi Borhade
StudentId : 109998404

November 2023

Contents

1	Linux Driver device	2
2	SELinux	11
2.1	What is SELinux?	11
2.2	Why should I use SELinux?	11
2.3	Is SELinux enabled on my system?	12
2.4	Where do I get these policies?	12

1 Linux Driver device

Write a basic Linux device driver program which explains about the kernel thread in Linux device driver.

```
1 #include <linux/kernel.h>           // Includes
   kernel-related functions and macros.
2 #include <linux/init.h>             // Includes
   module initialization and exit macros.
3 #include <linux/module.h>           // Includes
   module-related macros and data structures.
4 #include <linux/kdev_t.h>           // Includes
   kernel device number types and macros.
5 #include <linux/fs.h>               // Includes
   file system-related structures and functions
   .
6 #include <linux/cdev.h>             // Includes
   character device-related structures and
   functions.
7 #include <linux/device.h>           // Includes
   device model structures and functions.
8 #include <linux/slab.h>             // Includes
   memory allocation functions like kcalloc().
9 #include <linux/uaccess.h>          // Includes
   functions for user space data access.
10 #include <linux/kthread.h>          // Includes
   kernel thread management functions and data
   structures.
11 #include <linux/sched.h>            // Includes
   task management functions and task_struct
   structure.
12 #include <linux/delay.h>            // Includes
   delay functions like msleep().
13 #include <linux/err.h>              // Includes
   error code definitions and error handling
   functions.
14
```

```

15 dev_t dev = 0;                                // Device
    numbers.
16 static struct class *dev_class;                // Device
    class.
17 static struct cdev etx_cdev;                    //
    Character device structure.
18
19 static int __init etx_driver_init(void); //
    Function for module initialization.
20 static void __exit etx_driver_exit(void); //
    Function for module exit.
21
22 static struct task_struct *etx_thread;          //
    Kernel thread structure.
23
24 /*
25 ** Function Prototypes
26 */
27 /***** Driver functions
    *****/
28 static int etx_open(struct inode *inode, struct
    file *file); // Function for opening the
    device.
29 static int etx_release(struct inode *inode,
    struct file *file); // Function for closing
    the device.
30 static ssize_t etx_read(struct file *filp, char
    __user *buf, size_t len, loff_t * off); //
    Function for reading from the device.
31 static ssize_t etx_write(struct file *filp,
    const char *buf, size_t len, loff_t * off);
    // Function for writing to the device.
32 /*****
33
34 int thread_function(void *pv); // Function for
    the kernel thread.
35

```

```

36  /*
37  ** Thread
38  */
39  int thread_function(void *pv)
40  {
41      int i=0;
42      while(!kthread_should_stop()) { // Run the
43          thread until it's told to stop.
44          pr_info("In EmbeTronicX Thread Function
45              %d\n", i++); // Log a message.
46          msleep(1000); // Sleep for 1000
47              milliseconds.
48      }
49      return 0;
50  }
51
52  /*
53  ** File operation structure
54  */
55  static struct file_operations fops =
56  {
57      .owner          = THIS_MODULE,
58      .read            = etx_read,
59      .write           = etx_write,
60      .open            = etx_open,
61      .release         = etx_release,
62  };
63
64  /*
65  ** This function will be called when we open the
66      Device file
67  */
68  static int etx_open(struct inode *inode, struct
69      file *file)
70  {
71      pr_info("Device File Opened...!!!\n");
72      // Log a message when the device is
73      opened.

```

```

67         return 0;
68     }
69
70     /*
71     ** This function will be called when we close
       the Device file
72     */
73     static int etx_release(struct inode *inode,
       struct file *file)
74     {
75         pr_info("Device File Closed...!!!\n");
       // Log a message when the device is
       closed.
76         return 0;
77     }
78
79     /*
80     ** This function will be called when we read the
       Device file
81     */
82     static ssize_t etx_read(struct file *filp,
83         char __user *buf, size_t len,
84         loff_t *off)
85     {
86         pr_info("Read function\n"); // Log a
       message when reading from the device.
87
88         return 0;
89     }
90
91     /*
92     ** This function will be called when we write
       the Device file
93     */
94     static ssize_t etx_write(struct file *filp,
95         const char __user *buf, size_t
       len, loff_t *off)

```

```

96         pr_info("Write Function\n"); // Log a
          message when writing to the device.
97         return len;
98     }
99
100     /*
101     ** Module Init function
102     */
103     static int __init etx_driver_init(void)
104     {
105         /*Allocating Major number*/
106         if((alloc_chrdev_region(&dev, 0, 1, "
          etx_Dev")) < 0){ // Allocate a major
          number for the device.
107             pr_err("Cannot allocate major
          number\n"); // Log an error
          if allocation fails.
108             return -1;
109         }
110         pr_info("Major = %d Minor = %d \n",MAJOR
          (dev), MINOR(dev)); // Log the major
          and minor numbers.
111
112         /*Creating cdev structure*/
113         cdev_init(&etx_cdev, &fops); //
          Initialize the character device
          structure with file operations.
114
115         /*Adding character device to the system
          */
116         if((cdev_add(&etx_cdev, dev, 1)) < 0){
          // Add the character device to the
          system.
117             pr_err("Cannot add the device to the
          system\n"); // Log an error if
          addition fails.
118             goto r_class;
119         }

```

```

120
121      /*Creating struct class*/
122      if(IS_ERR(dev_class = class_create(
          THIS_MODULE, "etx_class"))){ //
          Create a device class.
123          pr_err("Cannot create the struct
              class\n"); // Log an error if
              creation fails.
124          goto r_class;
125      }
126
127      /*Creating device*/
128      if(IS_ERR(device_create(dev_class, NULL,
          dev, NULL, "etx_device"))){ //
          Create the device.
129          pr_err("Cannot create the Device \n
              "); // Log an error if creation
              fails.
130          goto r_device;
131      }
132
133      etx_thread = kthread_create(
          thread_function, NULL, "eTx Thread");
          // Create a kernel thread.
134      if(etx_thread) {
135          wake_up_process(etx_thread); //
          Start the kernel thread.
136      } else {
137          pr_err("Cannot create kthread\n");
          // Log an error if thread
          creation fails.
138          goto r_device;
139      }
140
141      pr_info("Device Driver Insert...Done!!!\n
          n"); // Log a success message.
142      return 0;
143

```

```

144
145 r_device:
146     class_destroy(dev_class); // Clean up
        class if there's an error.
147 r_class:
148     unregister_chrdev_region(dev, 1); //
        Unregister the allocated major/minor
        numbers.
149     cdev_del(&etx_cdev); // Delete the
        character device.
150     return -1;
151 }
152
153 /*
154 ** Module exit function
155 */
156 static void __exit etx_driver_exit(void)
157 {
158     kthread_stop(etx_thread); // Stop the
        kernel thread.
159     device_destroy(dev_class, dev); //
        Destroy the device.
160     class_destroy(dev_class); // Destroy
        the device class.
161     cdev_del(&etx

```

```

1      /* Add new data buffer and related variables */
2  #define BUF_SIZE 1024
3  static char device_buffer[BUF_SIZE]; // Data buffer
    to store information to be read or written.
4
5  /* ... Other code ... */
6
7  /*
8  ** This function will be called when we read the
    Device file
9  */
10 static ssize_t etx_read(struct file *filp, char
    _user *buf, size_t len, loff_t *off)
11 {
12     int bytes_to_copy, remaining_bytes;
13
14     if (*off >= BUF_SIZE) {
15         /* End of file */
16         return 0;
17     }
18
19     remaining_bytes = BUF_SIZE - *off;
20     bytes_to_copy = min(remaining_bytes, len);
21
22     if (copy_to_user(buf, device_buffer + *off,
        bytes_to_copy) != 0) {
23         pr_err("Data copy to user failed\n");
24         return -EFAULT; // Error code for failed
            memory copy.
25     }
26
27     *off += bytes_to_copy;
28     return bytes_to_copy;
29 }
30
31 /*
32 ** This function will be called when we write the

```

```

Device file
33 */
34 static ssize_t etx_write(struct file *filp, const
    char __user *buf, size_t len, loff_t *off)
35 {
36     int bytes_to_copy, remaining_bytes;
37
38     if (*off >= BUF_SIZE) {
39         /* End of file */
40         return -ENOSPC; // Error code for no space
            left.
41     }
42
43     remaining_bytes = BUF_SIZE - *off;
44     bytes_to_copy = min(remaining_bytes, len);
45
46     if (copy_from_user(device_buffer + *off, buf,
        bytes_to_copy) != 0) {
47         pr_err("Data copy from user failed\n");
48         return -EFAULT; // Error code for failed
            memory copy.
49     }
50
51     *off += bytes_to_copy;
52     return bytes_to_copy;
53 }

```

2 SELinux

2.1 What is SELinux?

SELinux, which stands for Security-Enhanced Linux, is a security framework for Linux operating systems. It was developed by the National Security Agency (NSA) and is now an integral part of many Linux distributions, including CentOS, Fedora, and Red Hat Enterprise Linux.

SELinux provides a mandatory access control (MAC) system that goes beyond the traditional discretionary access control (DAC) model used in most Unix-like operating systems. In a DAC system, permissions are assigned to users and groups, and files and processes inherit the permissions of their owners. This can lead to security vulnerabilities if a process is compromised, as it may have more access than necessary.

2.2 Why should I use SELinux?

It makes your Linux System in general more secure. Here are some examples :

1. **Mandatory Access Control:** SELinux enforces a mandatory access control (MAC) policy, which is more stringent than the traditional discretionary access control (DAC) used in Linux. This means that processes and users have only the minimum necessary privileges to perform their tasks. This can help mitigate the impact of security vulnerabilities and reduce the attack surface of the system.
2. **Least-Privilege Principle:** SELinux enforces the principle of least privilege, ensuring that processes and users have access only to the resources and actions required for their specific roles or tasks. This reduces the risk of unauthorized access or unintended consequences resulting from excessive permissions.
3. **Fine-Grained Control:** SELinux provides fine-grained control over system resources and processes. You can define specific policies that dictate what actions are allowed for each process, file, and resource, giving you precise control over security configurations.

2.3 Is SELinux enabled on my system?

```
1      sestatus
2
3 OR
4
5      getenforce
6
7      This command will tell you which setting of
        SELinux is active
8      1) Enforcing SELinux - is active and enforcing
9      2) Permissive - is active but not enforcing (
        only writes a log)
10     3) Disabled - SELinux is off
```

2.4 Where do I get these policies?

```
1      In SELinux, the policy consists of multiple
        files that work together to define security
        rules, object mappings, and interfaces. These
        files are processed and compiled into a
        single policy file that can be loaded into
        the kernel to enforce security.
2
3      Here's a brief overview of the key components of
        an SELinux policy:
4
5      Rule File (Type Enforcement File): This file
        contains rules and definitions for access
        controls. It defines how different types (e.g
        ., processes, files, and resources) can
        interact with each other and what actions are
        allowed or denied. The rule file typically
        has a ".te" extension.
6
7      Mapping File (File Contexts File): This file
        defines the mapping between file paths and
```

security labels (contexts). It associates labels with files and directories, specifying what context should be applied to various objects. The mapping file is usually named "file_contexts" and is used to ensure that files are labeled correctly.

8

9

Interface File (Optional): Interface files specify the management of SELinux policy modules. These files help control and load policy modules, allowing administrators to enable or disable specific parts of the policy. They are typically used for managing policy components. Interface files are optional and may have a ".if" extension.